

# Big Data and Automated Content Analysis (6EC)

Week 4: »Processing textual data«

Monday

---

Anne Kroon  
a.c.kroon@uva.nl

November 17, 2025

UvA RM Communication Science

# Today

Basic string operations

Regular expressions

What is a regexp?

Using a regexp in Python

The bag-of-words (BOW) model

General idea

A cleaner BOW representation

Better tokenization

Stopword removal

Pruning

Stemming and lemmatization

The order of preprocessing steps

This week, we will get a general overview of working with textual data. This knowledge will help you to get started with cool automated content analyses techniques – which we will start with next week.

Basic string operations  
oooooooo

Regular expressions  
oooooooooooooooo

The BOW  
oooooooooooooooooooooooooooooooooooooooooooooooo

Take-home exam  
ooo

At the end of the class, i'll distribute the  
take-home exam.

## Basic string operations

---















4



6













## Pandas string methods as alternative

If you select column with strings from a pandas dataframe, pandas offers a collection of string methods (via `.str.`) that largely mirror standard Python string methods:

```
1 df['newcolumnwithresults'] = df['columnwithtext'].str.count("bla")
```

## To pandas or not to pandas for text?

Partly a matter of taste.

Not-too-large dataset with a lot of extra columns? Advanced statistical analysis planned? Sounds like pandas.

It's mainly a lot of text? Wanna do some machine learning later on anyway? It's large and (potentially) messy? Doesn't sound like pandas is a good idea.

# Regular expressions

---

# Regular expressions

---

What is a regexp?











# Basic regexp elements

## Alternatives

`[TtFf]` matches either T or t or F or f

`Twitter|Facebook` matches either Twitter or Facebook

`.` matches any character

## Repetition

`?` the expression before occurs 0 or 1 times

`*` the expression before occurs 0 or more times

`+` the expression before occurs 1 or more times

# Basic regexp elements

## Alternatives

`[TtFf]` matches either T or t or F or f

`Twitter|Facebook` matches either Twitter or Facebook

`.` matches any character

## Repetition

`?` the expression before occurs 0 or 1 times

`*` the expression before occurs 0 or more times

`+` the expression before occurs 1 or more times





## regex quiz

Which words would be matched?

1. [Pp]ython
2. [A-Z]+
3. RT ? : ? @[a-zA-Z0-9]+





# Regular expressions

---

Using a regexp in Python



# How to use regular expressions in Python

## The module `re`\*

`re.findall("[Tt]witter|[Ff]acebook", testo)` returns a list with all occurrences of Twitter or Facebook in the string called `testo`

`re.findall("[0-9]+[a-zA-Z]+", testo)` returns a list with all words that start with one or more numbers followed by one or more letters in the string called `testo`

`re.sub("[Tt]witter|[Ff]acebook", "a social medium", testo)` returns a string in which all occurrences of Twitter or Facebook are replaced by "a social medium"

Use the less-known but more powerful module `regex` instead to support all dialects used in the book







## Example 1: Counting actors

```
1  import re
2  import csv
3  from glob import glob
4  import os
5
6  filenames = glob("/home/articles/*.txt")
7  results = []
8
9  for filepath in filenames:
10     with open(filepath, encoding="utf-8") as f:
11         text = f.read().replace('\n', ' ') # Read file + replace
12         ↪ newlines with spaces
13
14         israel_mentions = len(re.findall(
15             r'Israel.*?(minister|politician|authority)', text,
16             ↪ re.IGNORECASE))
17         palestine_mentions = len(re.findall(r'palest', text,
18             ↪ re.IGNORECASE))
19
20         results.append((
21             os.path.basename(filepath),
22             israel_mentions,
23             palestine_mentions
24         ))
```

## Example 2: Parsing semi-structured data

If your data look like this, you can loop over the lines and use regular expressions to extract the info you need!

```
1           All Rights Reserved
2
3           2 of 200 DOCUMENTS
4
5           De Telegraaf
6
7           21 maart 2014 vrijdag
8
9 Brussel bereikt akkoord aanpak probleembanken;
10 ECB krijgt meer in melk te brokkelen
11
12 SECTION: Finance; Blz. 24
13 LENGTH: 660 woorden
14
15 BRUSSEL Europa heeft gisteren op de valreep een akkoord bereikt
16 over een saneringsfonds voor banken. Daarmee staat de laatste
```



# Practice yourself!

Take some time to write some regular expressions. Write a script that

- extracts URLs from a list of strings
- removes everything that is not a letter or number from a list of strings

(first develop it for a single string, then scale up)

More tips:

<https://regex101.com/>

<http://www.regexpattern.com/>

# The BOW

---

# The BOW

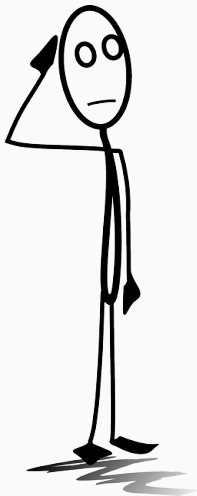
---

General idea





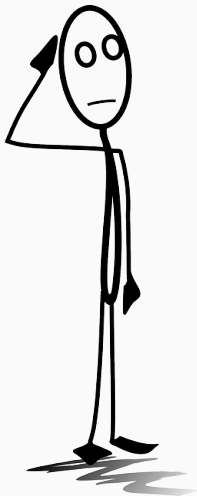




*What can you do with such a matrix?  
Why would you want to represent a  
collection of texts in such a way?*







*But are all terms equally important?*

## The cell entries: raw counts versus tf·idf scores

- In the example, we entered simple counts (the “term frequency”)
- But does a word that occurs in almost all documents contain much information?
- And isn’t the presence of a word that occurs in very few documents a pretty strong hint?
- Solution: Weigh by *the number of documents in which the term occurs at least once* (the “document frequency”)

⇒ we multiply the “term frequency” (tf) by the inverse document frequency (idf)

(usually with some additional logarithmic transformation and normalization applied, see [https://scikit-learn.org/stable/modules/generated/sklearn.feature\\_extraction.text.TfidfTransformer.html](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfTransformer.html))



## The cell entries: raw counts versus tf·idf scores

- In the example, we entered simple counts (the “term frequency”)
- But does a word that occurs in almost all documents contain much information?
- And isn’t the presence of a word that occurs in very few documents a pretty strong hint?
- **Solution:** Weigh by *the number of documents in which the term occurs at least once* (the “document frequency”)

⇒ we multiply the “term frequency” (tf) by the inverse document frequency (idf)

(usually with some additional logarithmic transformation and normalization applied, see [https://scikit-learn.org/stable/modules/generated/sklearn.feature\\_extraction.text.TfidfTransformer.html](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfTransformer.html))







# The BOW

---

A cleaner BOW representation



## Room for improvement

**tokenization** How do we (best) split a sentence into tokens  
(terms, words)?

**pruning** How can we remove unnecessary words?

**lemmatization** How can we make sure that slight variations of the  
same word are not counted differently?

# OK, good enough, perfect?

## .split()

- space → new word
- no further processing whatsoever
- thus, only works well if we do a preprocessing ourselves (e.g., remove punctuation)

```
1 docs = ["This is a text", "I haven't seen John's derring-do. Second  
    sentence!"]  
2 tokens = [d.split() for d in docs]
```

```
1 [['This', 'is', 'a', 'text'], ['I', "haven't", 'seen', "John's", 'derring-do.', 'Second', '  
    sentence!']]
```

# OK, good enough, perfect?

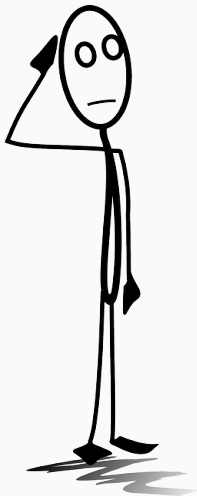
## Tokenizers from the NLTK package

- multiple improved tokenizers that can be used instead of `.split()`
- e.g., Treebank tokenizer:
  - split standard contractions ("don't")
  - deals with punctuation
  - BUT: Assumes lists of *sentences*.
- Solution: Build an own (combined) tokenizer (next slide)!

# OK, good enough, perfect?

```
1 import nltk
2 import re
3
4 class MyTokenizer:
5     def tokenize(self, text):
6         tokenizer = nltk.tokenize.TreebankWordTokenizer()
7         result = []
8         word = r"\w"
9         for sent in nltk.sent_tokenize(text):
10             tokens = tokenizer.tokenize(sent)
11             tokens = [t for t in tokens
12                       if re.search(word, t)]
13             result += tokens
14         return result
15
16 mytokenizer = MyTokenizer()
17 tokens = [mytokenizer.tokenize(d) for d in docs]
```

```
1 [['This', 'is', 'a', 'text'], ['I', 'have', "n't", 'seen', 'John', "'s", 'derring-do', 'Second',
    'sentence']]
```



*Can you (try to) explain the code?*

OK, so we can tokenize with a list comprehension (and that's often a good idea!). But what if we want to *directly* get a DTM instead of lists of tokens?

# OK, good enough, perfect?

## scikit-learn's CountVectorizer (default settings)

- applies lowercasing
- deals with punctuation etc. itself
- minimum word length  $> 1$
- more technically, tokenizes using this regular expression:

`r"(?u)\b\w\w+\b"`<sup>2</sup>

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 cv = CountVectorizer()
3 dtm_sparse = cv.fit_transform(docs)
```

---

<sup>2</sup>?u = support unicode, \b = word boundary

# OK, good enough, perfect?

## CountVectorizer supports more

- stopword removal
- custom regular expression
- or even using an external tokenizer
- ngrams instead of unigrams

see

[https://scikit-learn.org/stable/modules/generated/sklearn.feature\\_extraction.text.CountVectorizer.html](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html)

## Best of both worlds

Use the Count vectorizer with the custom NLTK-based external tokenizer we created before! `cv = CountVectorizer(tokenizer=mytokenizer.tokenize)`



# OK, good enough, perfect?

## CountVectorizer supports more

- stopword removal
- custom regular expression
- or even using an external tokenizer
- ngrams instead of unigrams

see

[https://scikit-learn.org/stable/modules/generated/sklearn.feature\\_extraction.text.CountVectorizer.html](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html)

## Best of both worlds

Use the Count vectorizer with the custom NLTK-based external tokenizer we created before! `cv = CountVectorizer(tokenizer=mytokenizer.tokenize)`

# Stopword removal

## What are stopwords?

- Very frequent words with little inherent meaning
- the, a, he, she, ...
- context-dependent: if you are interested in gender, he and she are no stopwords.
- Many existing lists as basis

When using the CountVectorizer, we can simply provide a stopwords list.

But we can also remove stopwords “by hand” of course using either a for loop (like we did for punctuation removal) or by modifying the tokenizer (try it!).

# General idea

- Idea behind both stopwords removal and tf-idf: too frequent words are uninformative
- (possible) downside stopwords removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf-idf: does not reduce number of features

Pruning: remove all features (tokens) that occur in less than X or more than X of the documents

## General idea

- Idea behind both stopwords removal and tf-idf: too frequent words are uninformative
- (possible) downside stopwords removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf-idf: does not reduce number of features

Pruning: remove all features (tokens) that occur in less than X or more than X of the documents

## General idea

- Idea behind both stopwords removal and tf-idf: too frequent words are uninformative
- (possible) downside stopwords removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf-idf: does not reduce number of features

Pruning: remove all features (tokens) that occur in less than X or more than X of the documents

# General idea

- Idea behind both stopwords removal and tf-idf: too frequent words are uninformative
- (possible) downside stopwords removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf-idf: does not reduce number of features

Pruning: remove all features (tokens) that occur in less than X or more than X of the documents

## CountVectorizer, only stopwords removal

```
1 from sklearn.feature_extraction.text import CountVectorizer,  
    TfidfVectorizer  
2 myvectorizer = CountVectorizer(stop_words=mystopwords)
```

CountVectorizer, other tokenization, stopwords removal (pay attention that stopwords list uses same tokenization!):

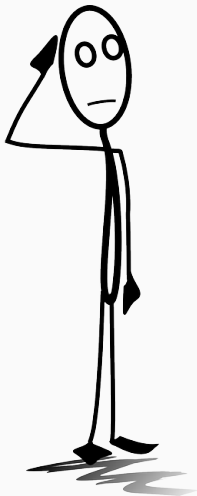
```
1 myvectorizer = CountVectorizer(tokenizer = TreebankWordTokenizer().  
    tokenize, stop_words=mystopwords)
```

Additionally remove words that occur in more than 75% or less than  $n = 2$  documents:

```
1 myvectorizer = CountVectorizer(tokenizer = TreebankWordTokenizer().  
    tokenize, stop_words=mystopwords, max_df=.75, min_df=2)
```

All together: tf-idf, explicit stopwords removal, pruning

```
1 myvectorizer = TfidfVectorizer(tokenizer = TreebankWordTokenizer().  
    tokenize, stop_words=mystopwords, max_df=.75, min_df=2)
```



*What is “best”? Which (combination of) techniques to use, and how to decide?*



# Stemming and lemmatization

- Stemming: reduce words to its stem by removing last part (drinking → drink)
- Lemmatization: find word that you would need to look up in a dictionary (drinking → drink, but also went → go)
- stemming is simpler than lemmatization
- lemmatization often better

Example below: tokenization and lemmatization with spacy in one go:

```
1 import spacy
2 nlp = spacy.load('en') # potentially you need to install the language
  model first
3 lemmatized_tokens = [[token.lemma_ for token in nlp(doc)] for doc in
  docs]
```

```
1 [['this', 'be', 'a', 'text'], ['-PRON-', 'have', 'not', 'see', 'John', "s", 'derring', '-', 'do',
  ', ', 'second', 'sentence', '!']]
```

# Stemming and lemmatization

- Stemming: reduce words to its stem by removing last part (drinking → drink)
- Lemmatization: find word that you would need to look up in a dictionary (drinking → drink, but also went → go)
- stemming is simpler than lemmatization
- lemmatization often better

Example below: tokenization and lemmatization with spacy in one go:

```
1 import spacy
2 nlp = spacy.load('en') # potentially you need to install the language
  model first
3 lemmatized_tokens = [[token.lemma_ for token in nlp(doc)] for doc in
  docs]
```

```
1 [['this', 'be', 'a', 'text'], ['-PRON-', 'have', 'not', 'see', 'John', "'s", 'derring', '-', 'do',
  ', .', 'second', 'sentence', '!']]
```

# The BOW

---

The order of preprocessing steps

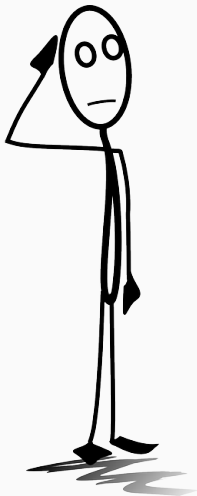
# Option 1

## Preprocessing only through Vectorizer

“Just use CountVectorizer or TfidfVectorizer with the appropriate options.”

- pro: No double work, efficient if your main goal is a sparse matrix (for ML?) anyway
- con: you cannot “see” the preprocessed texts





*How would you do it?*

Sometimes, I go for Option 2 because

- I like to inspect a sample of the documents
- I can re-use the cleaned docs irrespective of the Vectorizer

But at other times, I opt of Option 1 instead because

- I want to systematically compare the effect of different choices in a machine learning pipeline (then I can simply vary the vectorizer instead of the data)
- I want to use techniques that are geared towards little or no preprocessing (deep learning)

# The BOW

---

How further?





## More NLP

**$n$ -grams** Consider using  $n$ -grams instead of unigrams

**collocations** *ngrams* that appear more frequently than expected

## POS-tagging grammatical function ("part-of-speech") of tokens

**NER** named entity recognition (persons, organizations, locations)



# Take-home exam

---

## Deadline: Sunday 23, 23:59

